Aiced Revision Panel: What Happened

Plain-English notes for explaining why the “Revised by Aiced” panel disappeared, and how it was fixed.

THE SHORT ANSWER

This was not an AI, database, or CSS problem. One JavaScript variable had the wrong name. The page successfully began drawing the Original article, then JavaScript stopped before it could draw the Revised by Aiced panel.

WHAT THE USER WAS SUPPOSED TO SEE

```
Ask Aiced for a revision
  -> Flask asks AI for a proposed change
  -> Flask validates the proposal but does not save it yet
  -> Browser displays Original on the left and Revised by Aiced on the right
  -> User chooses Accept Changes or Discard
```

WHAT ACTUALLY HAPPENED

1. The AI revision request succeeded.
2. The browser received the current article and proposed revision.
3. The browser started rendering the left-side Original article.
4. It calculated which article blocks had changed.
5. That calculation referenced proposedBlocks, but the code had created the variable as revisedBlocks.
6. JavaScript stopped before it appended the right-side Revised by Aiced panel.

THE EXACT BUG

The comparison renderer created:

```
const revisedBlocks = revisedArticle.contentBlocks || [];
```

But it later tried to use:

```
proposedBlocks[proposedIndex]
```

proposedBlocks did not exist. The correction uses:

```
revisedBlocks[proposedIndex]
```

WHY IT WAS CONFUSING

- It looked like a CSS/layout problem.
- Flask returned a successful response, so server logs looked normal.
- The Original side rendered before the error, so only the right side looked broken.
- The feature crosses AI response, validation, browser state, comparison logic, and DOM rendering.

WHAT CHANGED

- A pendingArticleRevision view model now holds the current article, proposed article, and changed fields.
- Normal article rendering and pending comparison rendering are separate paths.
- An Article revision keeps the Original / Revised by Aiced comparison visible during workspace rerenders.
- CTA-only, Campaign-only, and Audience-only revisions explicitly say that the article itself is unchanged.
- Accept still uses the existing workflow. Discard still changes no saved data.

- Publishing and sending remain separate human actions.

TESTS ADDED

- Content-block revision shows both columns.
- Title-only revision shows both columns.
- CTA-only revision shows a non-article review notice.
- Discard restores the normal article.
- Accept applies the revision to the browser proposal before handoff.
- Rerendering keeps exactly one revised panel visible.

ANALOGY

Imagine two people setting up a before-and-after display. The first puts the “before” page on the wall. The second is supposed to hang the “after” page, but their instruction sheet refers to a tool name that does not exist. They stop. The first page remains on the wall, but the second never gets hung.

Scope: frontend revision display only. No database schema, AI prompt, publishing, recipient selection, or email-sending behavior changed.